

◆ .NET Core / C# Advanced Questions & Answers

1. What's the difference between .NET Framework, .NET Core, and .NET 6/7 (modern .NET)?

Answer:

- **.NET Framework:** Windows-only, older, supports WebForms, WCF, ASP.NET MVC.
 - **.NET Core:** Cross-platform, lightweight, modular, supports microservices.
 - **.NET 5/6/7+ (.NET unified platform):** Unified runtime combining .NET Framework, Core, Xamarin, etc. Supports modern workloads with long-term support (LTS).
-

2. What is the difference between synchronous and asynchronous programming in C#?

Answer:

- **Synchronous:** Tasks execute one after another, blocking the thread.
- **Asynchronous (async/await):** Allows non-blocking operations, freeing threads for other tasks (e.g., I/O, DB calls).
- Example:

```
public async Task<string> GetDataAsync() { HttpClient client = new HttpClient(); return  
await client.GetStringAsync("https://api.com/data"); }
```

3. Explain Dependency Injection (DI) in .NET Core.

Answer:

- DI is a design pattern to achieve **loose coupling** between classes.
- .NET Core has a built-in IoC container (IServiceCollection).
- Example:

```
public interface IRepository { void Save(); } public class SqlRepository : IRepository {  
public void Save() { } } public class HomeController { private readonly IRepository _repo;  
public HomeController(IRepository repo) { _repo = repo; } }
```

- Registered in Startup.cs :

```
services.AddScoped<IRepository, SqlRepository>();
```

4. What is the difference between IEnumerable , IQueryable , and List in C#?

Answer:

- **IEnumerable:** In-memory, forward-only iteration, LINQ-to-Objects.
 - **IQueryable:** Extends IEnumerable, allows LINQ-to-SQL/Entity Framework queries (executed at DB side).
 - **List:** Concrete collection class, stores objects in memory.
-

5. What is middleware in .NET Core?

Answer:

- Middleware is software components in the request/response pipeline.
- Each middleware decides whether to process the request or pass it to the next.
- Example in Startup.cs :

```
app.UseRouting(); app.UseAuthentication(); app.UseAuthorization();  
app.UseEndpoints(endpoints => endpoints.MapControllers());
```

6. What are Delegates and Events in C#?

Answer:

- **Delegate:** Type-safe function pointer. Example:

```
public delegate void MyDelegate(string msg);
```

- **Event:** Wrapper around delegate, used for publish-subscribe. Example:

```
public event MyDelegate OnMessage;
```

- Events ensure encapsulation (only publisher can invoke).
-

7. What are the different lifetimes of services in .NET Core DI?

Answer:

- **Transient:** Created every time requested.
 - **Scoped:** Created once per request.
 - **Singleton:** Created once per application lifetime.
-

8. What is Entity Framework Core and how is it different from EF6?

Answer:

- EF Core: Lightweight, cross-platform ORM for .NET Core.
 - Supports LINQ queries, migrations, change tracking.
 - Differences:
 - EF6 is Windows-only, mature, but heavy.
 - EF Core is cross-platform, lightweight, faster, but initially had fewer features (now improved).
-

9. What is the difference between Task , ValueTask , and Thread ?

Answer:

- **Thread**: Actual OS-managed thread. Expensive to create.
 - **Task**: Higher-level abstraction for async operations (can use thread pool).
 - **ValueTask**: Optimized for cases where result is available synchronously (avoids allocations).
-

10. How would you design a scalable Web API in .NET Core?

Answer:

- Use **RESTful principles** (stateless, resource-based URLs).
- Implement **async/await** for I/O operations.
- Use **Dependency Injection** for service management.
- Enable **caching** (in-memory, Redis, response caching).
- Secure with **JWT tokens / OAuth2**.
- Use **Swagger/OpenAPI** for documentation.
- Deploy in **containers (Docker/Kubernetes)** for scalability.